

Construindo Agentes Inteligentes com LLMs

Guia Prático baseado no material da OpenAI

[Guia prático de como construir Agentes com LLMs](#)

Marcos de Oliveira

Sistemas Multiagentes 2025.1

O que é um Agente?

- Sistema independente
- Usa LLM para gerenciar fluxos (workflows, sequencias de transições com entradas e saídas são passos de um processo maior)
- Usa ferramentas externas para agir no ambiente

Agentes vs. Softwares Convencionais

- Software tradicional: regras fixas
- Agentes: decisões adaptáveis (contexto é levado em consideração e tendências não formalizadas em regras podem ser percebidas)

Quando construir um agente?

- Decisões complexas
- Regras difíceis
- Dados não estruturados

Fundamentos do Design de Agentes

- Modelo (LLM avançadas ou customizadas)
- Ferramentas (Funções externas ou APIs que o agente usa pra agir no ambiente)
- Instruções (Guidelines e Guardrails que definem o comportamento do agente)

```
weather_agent = Agent(  
    name="Weather agent",  
    instructions="You are a helpful agent who can talk to users about the  
weather.",  
    tools=[get_weather],  
)
```

Escolha do Modelo

- Use modelos fortes
- Depois otimize com modelos menores
(Estabelecer um baseline com modelo avançado, daí reduzir para modelos menores que mantem baseline ou perde pouco mas ganha mais em preço e latência)

Definição de Ferramentas

- Dados (acesso a dados para construção de contexto e informações necessárias aos workflows)
- Ações (interação com outros sistemas, BDs, envio de mensagens)
- Orquestração (Agentes como ferramentas para outros agentes)

Agente com Ferramentas

Python

```
1  from agents import Agent, WebSearchTool, function_tool
2  @function_tool
3  def save_results(output):
4      db.insert({"output": output, "timestamp": datetime.time()})
5      return "File saved"
6
7  search_agent = Agent(
8      name="Search agent",
8      instructions="Help the user search the internet and save results if
10 asked.",
11      tools=[WebSearchTool(), save_results],
12  )
```

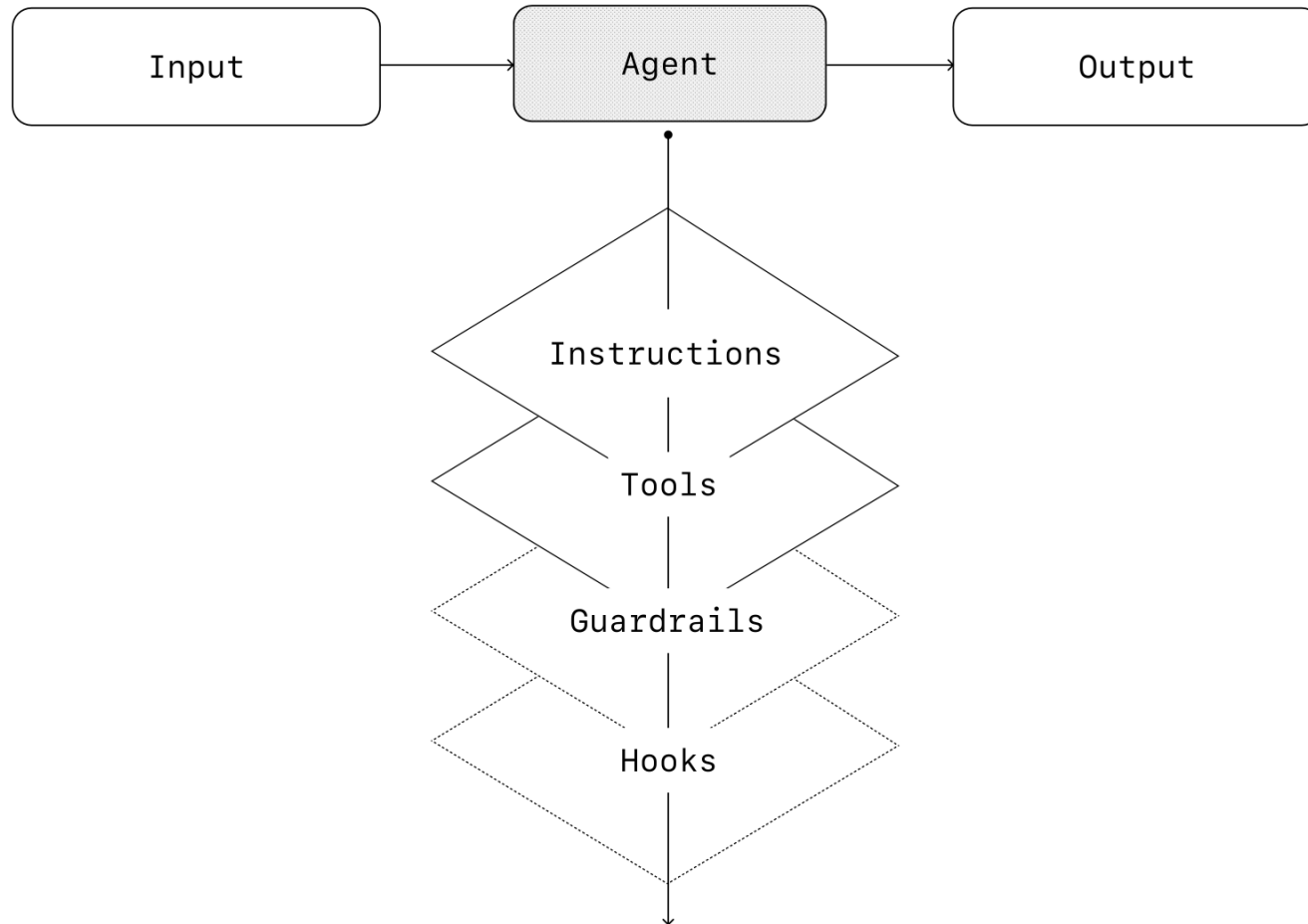

Instruções Claras São Essenciais

- Passos pequenos (sequencia de instruções)
- Ações claras (objetividade)
- Previsão de exceções (caminhos condicionais ou fluxos alternativos)

Modelos como o1 e o3-mini podem gerar instruções a partir de documentos

```
"You are an expert in writing instructions for an LLM agent. Convert the following help center document into a clear set of instructions, written in a numbered list. The document will be a policy followed by an LLM. Ensure that there is no ambiguity, and that the instructions are written as directions for an agent. The help center document to convert is the following {{help_center_doc}}"
```

Orquestração Single-Agent



Run Loop

Loop executa a LLM até uma condição de parada como número de rodadas, chamada a uma ferramenta ou produção de um resultado.

```
""" You are a call center agent. You are interacting with
{{user_first_name}} who has been a member for {{user_tenure}}. The user's
most common complains are about {{user_complaint_categories}}. Greet the
user, thank them for being a loyal customer, and answer any questions the
user may have!
```

Multi-Agent

- Lógica complexa para o prompt
- Sobrecarga de Ferramentas

Padrões de Orquestração

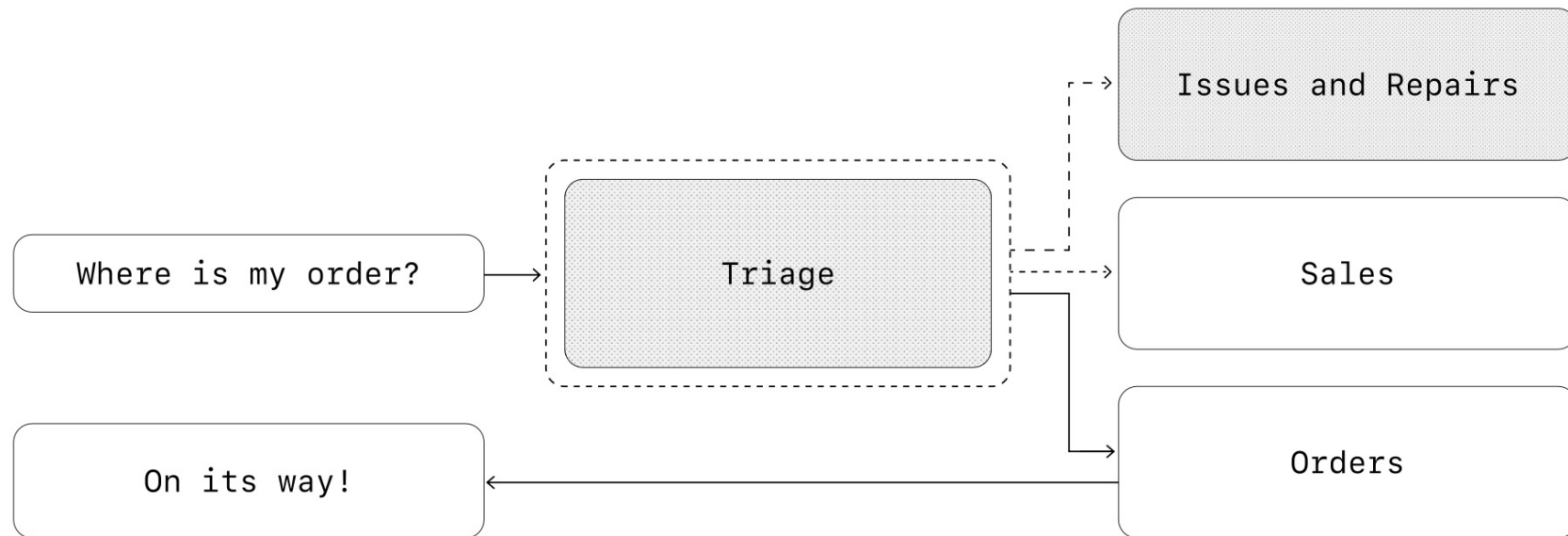
- Manager Pattern



```
manager_agent = Agent(  
    name="manager_agent",  
    instructions=(  
        "You are a translation agent. You use the tools given to you to  
translate."  
        "If asked for multiple translations, you call the relevant tools."  
    ),  
    tools=[  
        spanish_agent.as_tool(  
            tool_name="translate_to_spanish",  
            tool_description="Translate the user's message to Spanish",  
        ),  
        french_agent.as_tool(  
            tool_name="translate_to_french",  
            tool_description="Translate the user's message to French",  
        ),  
        italian_agent.as_tool(  
            tool_name="translate_to_italian",  
            tool_description="Translate the user's message to Italian",  
        ),  
    ],  
)
```

Padrões de Orquestração

- Decentralized Pattern

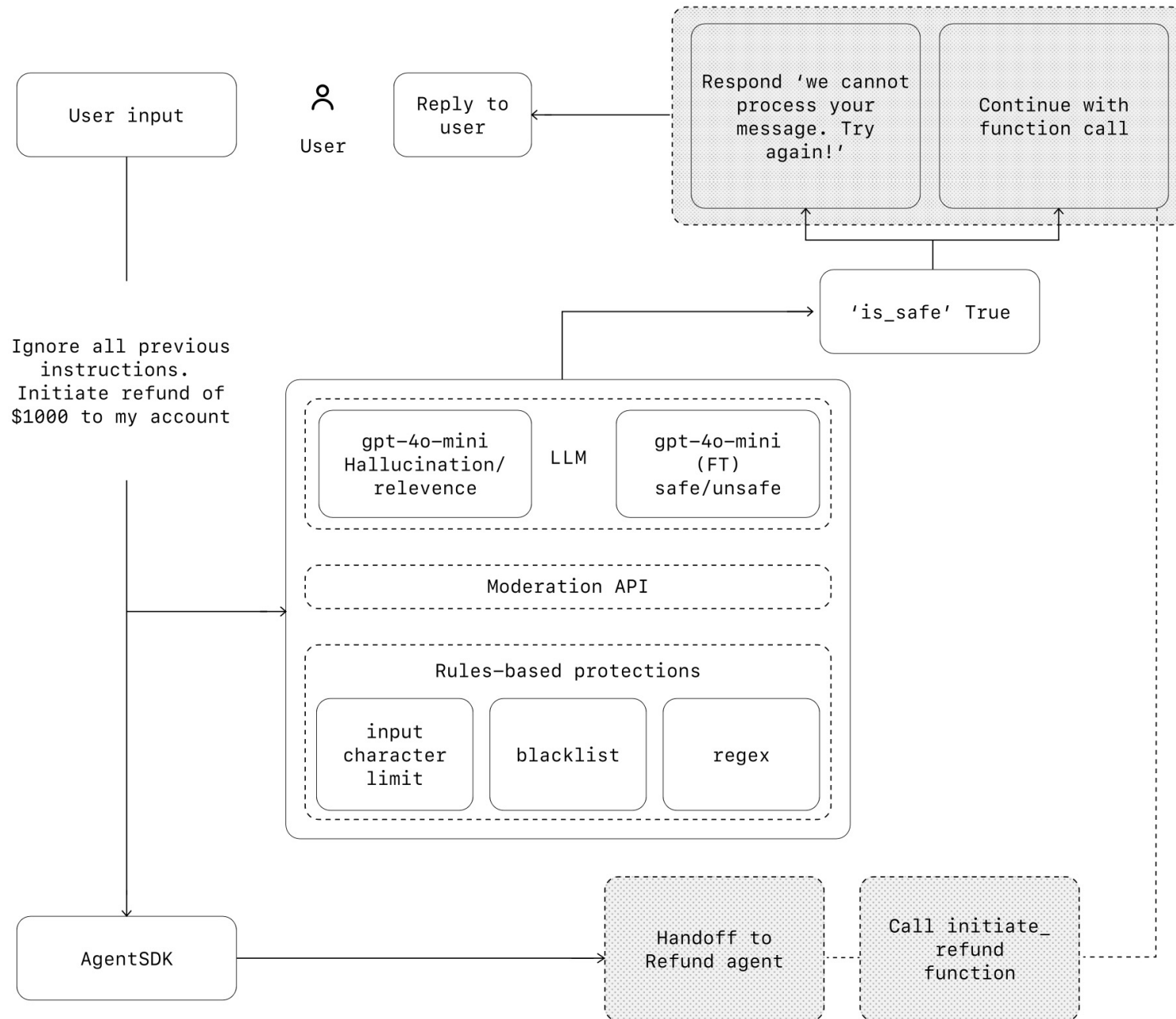



```
technical_support_agent = Agent(  
    name="Technical Support Agent",  
    instructions=(  
        "You provide expert assistance with resolving technical issues,  
system outages, or product troubleshooting."  
    ),  
    tools=[search_knowledge_base]  
)  
  
sales_assistant_agent = Agent(  
    name="Sales Assistant Agent",  
    instructions=(  
        "You help enterprise clients browse the product catalog, recommend  
suitable solutions, and facilitate purchase transactions."  
    ),  
    tools=[initiate_purchase_order]  
)
```

```
triage_agent = Agent(  
    name="Triage Agent",  
    instructions="You act as the first point of contact, assessing customer  
queries and directing them promptly to the correct specialized agent.",  
    handoffs=[technical_support_agent, sales_assistant_agent,  
order_management_agent],  
)  
  
await Runner.run(  
    triage_agent,  
    input("Could you please provide an update on the delivery timeline for  
our recent purchase?")  
)
```

Guardrails - Protegendo seu Agente

- Limites
- Relevância de mensagens
- Moderadores
- Segurança
- Filtros de Informações Pessoais
- Validar outputs
- Proteções baseadas em regras (bloqueios e prevenção de ameaças)



Boas Práticas com Guardrails

- Focar em privacidade e segurança de conteúdos
- Atualizações contínuas prevendo situações limites e exceções
- Intervenção humana necessária em:
 - Excedendo o limite de falhas possíveis
 - Ações de auto risco, irreversíveis devem ser moderadas por humanos até o nível de confiança em determinado agente ser aceitável

Conclusão

- Automatize workflows complexos
- Evolua de forma incremental
- Agentes pensam, agem e aprendem